

Video Streaming and Content Distribution Networks (CDNs)

Introduction

Video streaming accounts for the majority of Internet traffic, with platforms like **Netflix**, **YouTube**, **Amazon Prime**, and **TikTok** dominating usage.

- By 2020, video traffic represented **80% of global Internet traffic** [?].
- Video streaming relies on **application-layer protocols**, **HTTP servers**, and **CDNs** to ensure smooth delivery.
- Major challenges: *high bitrates*, *bandwidth fluctuation*, *scalability*.

Internet Video

Characteristics of Video Media

- A video = sequence of images (**24–30 fps typical**).
- Each frame: array of **pixels** (luminance + color).
- Compression reduces bitrate while trading off quality.

Definition

Compressed Video Bitrates:

- Low quality: ~ 100 kbps
- HD movie: 4 Mbps
- 4K video: > 10 Mbps

Networking Perspective

- Continuous playback requires **average throughput $\geq$ video bitrate**.
- Example: A 2 Mbps, 67-min video $\approx$ 1 GB storage.
- Performance metric: **End-to-end throughput**.

HTTP Streaming and DASH

HTTP Streaming

- Video stored as **ordinary file** on HTTP server.
- Client requests via HTTP **GET**; server sends file as fast as possible.
- Client buffers video, starts playback once threshold reached.

Limitation

- All clients receive same encoding regardless of bandwidth.

Dynamic Adaptive Streaming over HTTP (DASH)

- Video encoded into **multiple versions** (different bitrates).
- Client dynamically requests **chunks** (few seconds each).
- High bandwidth $\Rightarrow$ high-quality chunks; low bandwidth $\Rightarrow$ low-quality chunks.
- Adaptation critical for **mobile users**.

Note

DASH uses a **manifest file**, which lists available versions and bitrates. Client requests segments using HTTP GET with byte ranges.

Content Distribution Networks (CDNs)

Motivation

Challenges of single data center approach:

1. Long paths $\Rightarrow$ bottlenecks $\Rightarrow$ playback freezes.
2. Repeated traffic wastes bandwidth (same video sent multiple times).
3. Single point of failure.

CDN Design

- Servers at multiple geographic locations.
- Stores cached copies of content.
- Directs users to optimal server for performance.

CDN Architectures

- **Enter Deep (Akamai)**: Small clusters inside access ISPs, closer to users.
- **Bring Home (Limelight)**: Large clusters at IXPs; lower management overhead.

Cluster Selection Strategies

- **Geo-location**: Closest cluster (may not reflect actual path quality).
- **Real-time measurements**: Delay/loss probes to LDNS.
- Tradeoff between accuracy and feasibility.

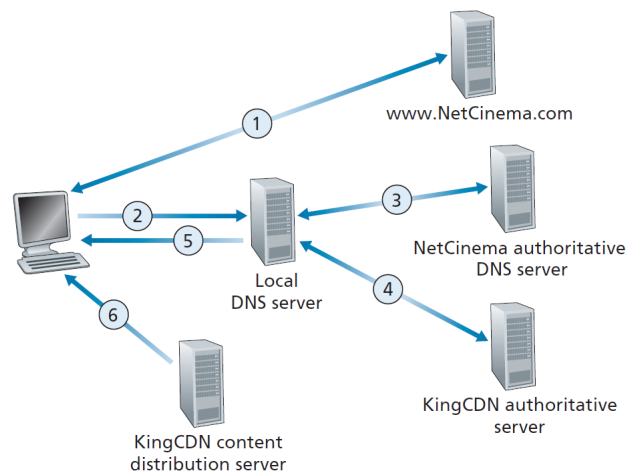


Figure 1: CDN deployment strategies: Enter Deep vs Bring Home

Case Studies

Netflix

Case Study

Netflix uses a hybrid of **Amazon Cloud** + **Private CDN**.

Amazon Cloud Responsibilities:

- Content ingestion and processing (multiple formats + bitrates).
- Upload processed versions to CDN.
- Runs website (login, billing, catalog, recommendations).

Private CDN (Open Connect):

- Servers in 200+ IXPs and residential ISPs.
- Push caching (videos distributed during off-peak).
- Entire Netflix library stored in large IXP racks; popular content in ISP racks.
- Client chooses chunks using DASH with $\sim 4s$ segments.

YouTube (Google)

Case Study

YouTube leverages Google's massive private CDN with both IXP and ISP clusters.

Characteristics:

- Uses **pull caching** and DNS redirection.

- Cluster chosen typically by lowest RTT, with load balancing.
- HTTP streaming with manual quality selection (no DASH).
- Uploaders also use HTTP to send videos to Google's data centers.
- Videos processed into multiple bitrates.

Summary

- Streaming requires sustained throughput $\geq$ video bitrate.
- HTTP streaming evolved into DASH for adaptive quality delivery.
- CDNs reduce latency, improve throughput, and scale video distribution.
- Netflix: Push caching + private CDN.
- YouTube: Pull caching + DNS-based Google CDN.